


DOCUMENTO DEL RETO

# Detección y clasificación de embarcaciones en imágenes satelitales

*SpaceLAB · Hackatón FAC+PUJ · GeoVision Colombia*

| | |
|---|---|
| **Fecha** | Sábado 7 de noviembre de 2026 · 7:00 a.m. – 5:30 p.m. |
| **Lugar** | Free Plaza, Edificio de Laboratorios · Pontificia Universidad Javeriana, Bogotá |
| **Organizan** | Facultad de Ingeniería PUJ · Fuerza Aeroespacial Colombiana (FAC) |
| **Modalidad** | Presencial, por equipos · un solo día |
| **Ganador** | El equipo con el mayor puntaje en la parte privada del conjunto oculto |

### El reto en una frase

Construya, en un día y con recursos limitados, un modelo de inteligencia artificial que **encuentre cada embarcación** en una imagen satelital, la **encierre en una caja** y diga **de qué tipo es** (5 clases).

# Contenido

1. Contexto y motivación

2. La tarea

3. Los datos

4. Formato de envío

5. Evaluación

6. Reglas

7. Agenda propuesta

8. Cómo empezar

Anexo · Glosario

# 1. Contexto y motivación

Colombia tiene costas sobre dos océanos y un extenso territorio marítimo que vigilar. Saber qué embarcaciones hay, dónde están y de qué tipo son es clave para la seguridad y la defensa, la lucha contra la pesca ilegal y el contrabando, la búsqueda y el rescate, y la gestión de puertos.

Las imágenes satelitales de muy alta resolución permiten observar grandes extensiones de mar y costa, pero revisarlas a mano es lento y costoso. La visión por computador puede automatizar esa tarea: un buen detector marca en segundos cientos de barcos que un analista tardaría horas en encontrar.

En este hackatón, los equipos construirán ese detector. El reto pone a prueba no solo la precisión del modelo, sino la capacidad de tomar buenas decisiones de ingeniería con tiempo y cómputo limitados.

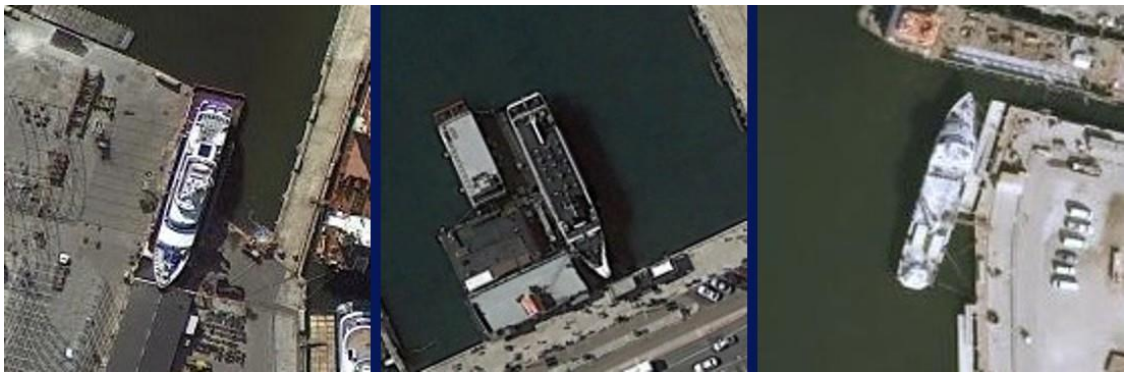
# 2. La tarea

Es un problema de **detección de objetos**: para cada imagen, el modelo debe producir una lista de cajas rectangulares (horizontales), cada una con una **clase** y una **confianza** entre 0 y 1.

| ID | Clase | Descripción |
|---|---|---|
| 0 | small_boat | Embarcaciones pequeñas: lanchas, botes, yates y embarcaciones de recreo. |
| 1 | cargo_ship | Buques de carga: portacontenedores, graneleros, tanqueros, barcazas de carga. |
| 2 | fishing_vessel | Barcos pesqueros. |
| 3 | warship | Buques militares: fragatas, destructores, portaaviones, buques de apoyo naval. |
| 4 | work_vessel | Embarcaciones de servicio: remolcadores, dragas, grúas flotantes, buques de trabajo. |

*Tabla 1. Clases del reto (el ID es el número que va en los archivos de predicción).*

Ejemplos de cada clase (recortes de imágenes de entrenamiento):



*Clase 0 · small_boat*



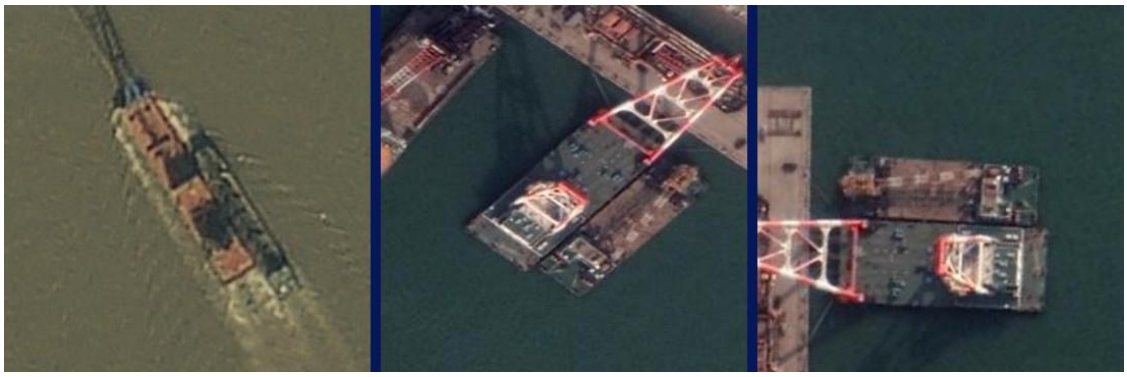
*Clase 1 · cargo_ship*



*Clase 2 · fishing_vessel*



*Clase 3 · warship*



*Clase 4 · work_vessel*

# 3. Los datos

El dataset reúne imágenes ópticas de muy alta resolución de cuatro colecciones públicas de teledetección (FAIR1M, ShipRSImageNet, VHRShips y xView), unificadas en las 5 clases del reto y en formato YOLO.

| Conjunto | Imágenes | Objetos | Uso |
|---|---|---|---|
| train | 5 926 | 22 155 | Entrenamiento (con etiquetas) |
| test | 801 | 3 964 | Validación local (con etiquetas) |
| oculto | 1 743 | 6 811 | Evaluación oficial (SIN etiquetas) |

*Tabla 2. Conjuntos de datos.*

| Objetos por clase | small | cargo | fishing | warship | work |
|---|---|---|---|---|---|
| train | 5 600 | 5 600 | 4 988 | 4 255 | 1 712 |
| test | 800 | 841 | 826 | 697 | 800 |

*Tabla 3. Distribución de clases. work_vessel es la clase minoritaria en train.*

## Características importantes

- **Objetos muy pequeños:** un barco ocupa en promedio ~0,3 % del área de la imagen, y muchos menos del 0,05 %. La resolución de entrada importa.
- **Escenas densas:** hay imágenes con más de 200 embarcaciones (puertos, astilleros, bases navales).
- **Tamaños variados:** de 600 a 1 500 píxeles por lado; todas en JPEG RGB.
- **Vista cenital:** los barcos aparecen en cualquier orientación; no hay un "arriba" natural.
- **Fuentes distintas:** cambian el sensor, la resolución y el estilo de anotación. El conjunto oculto tiene la misma mezcla de fuentes que test.

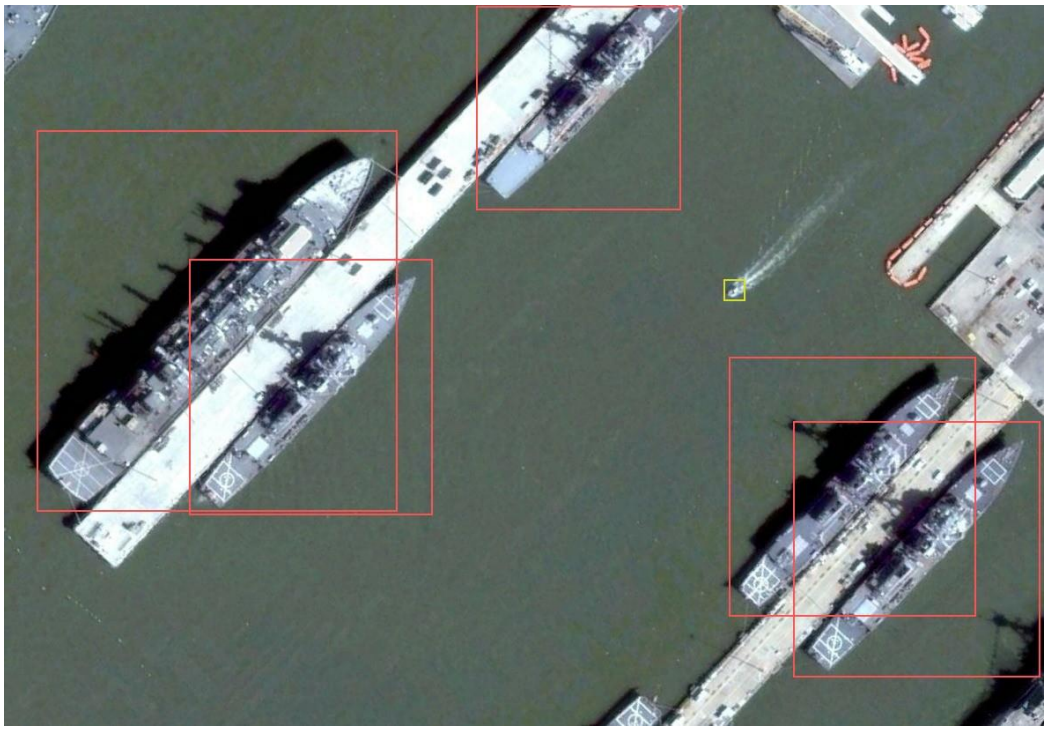


*Figura 1. Imagen de entrenamiento con sus etiquetas: buques militares (rojo) y una embarcación pequeña (amarillo).*

### Formato de las etiquetas (YOLO)

Cada imagen `nombre.jpg` tiene un archivo `nombre.txt` con una línea por objeto:

```
clase  x_centro  y_centro  ancho  alto
```

Las coordenadas están normalizadas entre 0 y 1 respecto al ancho y alto de la imagen. Ejemplo:

```
1 0.436667 0.944000 0.141333 0.112000
```

### Qué recibe cada equipo

Un archivo `spacelab_dataset.zip` (~1,8 GB) con esta estructura:

```
spacelab_dataset/
├── data.yaml          (listo para Ultralytics: train=train, val=test)
├── images/train/      labels/train/
├── images/test/       labels/test/
└── images/oculto/     (solo imágenes, nombres sl_xxxxxxxx.jpg)
```

Además: dos baselines para empezar, YOLO26 (`baseline_colab.ipynb` o `entrenar_baseline.py`) y Faster R-CNN (`baseline_frcnn_colab.ipynb` o `entrenar_baseline_frcnn.py`); el evaluador oficial `evaluar.py`, y `formato_envio.py`, que arma y revisa el ZIP de envío desde cualquier modelo.

## 4. Formato de envío

Un envío es un **archivo ZIP** con un `.txt` por cada imagen de `images/oculto`, con el mismo nombre de la imagen. Cada línea es una detección:

```
clase  x_centro  y_centro  ancho  alto  confianza
```

Ejemplo de `sl_3fa9c2d1.txt`:

```
3 0.512300 0.401200 0.210000 0.088000 0.93
0 0.120400 0.775000 0.015000 0.020100 0.41
1 0.880000 0.300500 0.150200 0.061000 0.07
```

- **Imagen sin archivo o archivo vacío** = el modelo no detectó nada en ella.
- **Máximo 300 detecciones por imagen;** si hay más, se usan las 300 de mayor confianza.
- **umbral.txt (opcional):** un número entre 0 y 1 (p. ej. `0.30`). Es el umbral de confianza con el que se calcula el F1 (ver sección 5). Si no se incluye, se usa 0,25. También se puede escribir en el formulario de envío.
- Es exactamente el formato de Ultralytics con `save_txt=True, save_conf=True`. Da igual si los .txt están en la raíz del ZIP o en una subcarpeta.
- Antes de enviar, pruebe su formato localmente con `evaluar.py` sobre el conjunto test, y revise el ZIP del oculto con `python formato_envio.py verificar --envio su_envio.zip --imagenes spacelab_dataset/images/oculto`.

# 5. Evaluación

> **Puntaje oficial**
>
> **Puntaje = 0,7 × mAP@0.5:0.95 + 0,3 × F1macro@0.5**
>
> Va de 0 a 1. Más alto es mejor. Desempate: mAP@0.5; si persiste, gana el envío más temprano.

### ¿Cuándo una detección es correcta?

Una caja predicha acierta (es un **verdadero positivo**) si tiene la **misma clase** que una caja real y su **IoU** (intersección sobre unión: el área compartida dividida por el área total que cubren las dos cajas) supera un umbral. Cada caja real solo puede emparejarse con una predicción; las predicciones se emparejan en orden de confianza, de mayor a menor. Las predicciones sobrantes son **falsos positivos** y las cajas reales no encontradas son **falsos negativos**.

### Componente 1 · mAP@0.5:0.95 (70 %)

La métrica estándar de detección (benchmark COCO), usada también en FAIR1M y en la mayoría de competencias. Para cada clase se calcula la **precisión promedio (AP)**: se ordenan todas las predicciones por confianza y se mide el área bajo la curva precisión–recall (interpolación de 101 puntos). Esto se repite

con 10 umbrales de IoU (0,50; 0,55; …; 0,95) y se promedia, y luego se promedia sobre las 5 clases (todas pesan igual).

- Premia encontrar todos los barcos, no generar falsas alarmas, clasificarlos bien y ajustar las cajas con precisión.
- Usa todas las cajas enviadas: enviar detecciones de baja confianza no penaliza el mAP.

### Componente 2 · F1macro@0.5 (30 %)

Mide qué tan útil sería el sistema en operación, donde hay que decidir qué alarmas mostrar al analista. Solo cuentan las cajas con **confianza ≥ umbral del equipo** (umbral.txt). Con IoU ≥ 0,5, para cada clase:

```
Precisión = VP/(VP+FP)   Recall = VP/(VP+FN)   F1 = 2·P·R/(P+R)
```

Luego se promedia el F1 de las 5 clases. Competencias de detección de barcos como Airbus Ship Detection (Kaggle) y xView3 usaron métricas de este tipo.

> **¿Por qué dos métricas?**
>
> El mAP es la medida más completa y comparable de un detector. Pero por sí sola no obliga a decidir: un modelo puede inundar cada imagen con cientos de cajas dudosas sin castigo.
>
> El F1 obliga a cada equipo a elegir un punto de operación, como en un sistema real. Combinar ambas premia los modelos que son buenos y además se pueden usar.

### Leaderboard en vivo: público y privado

El conjunto oculto está dividido al azar (estratificado por fuente) en dos partes que nadie conoce: **pública (~30 %, 523 imágenes)** y **privada (~70 %, 1 220 imágenes)**.

- Durante el día, cada envío se evalúa al instante y el leaderboard muestra **solo el puntaje público**.
- Para el resultado final se toma de cada equipo **su envío con mejor puntaje público**, y se ordena por su **puntaje privado**. Se revela al cierre.
- Esto evita que gane quien ajuste su modelo al azar de la parte pública. Lo que cuenta es generalizar: confíe en su validación local con test.

Dirección del leaderboard: **http://192.168.0.100:8501**. Cada equipo recibe un **token** para enviar y ver su historial.

## 6. Reglas

### Equipos y envíos

- Equipos de 2 a 4 personas. Cada persona pertenece a un solo equipo.

- Máximo **15 envíos por equipo**, con al menos **20 minutos** entre envíos. Los envíos con error de formato no cuentan.
- Cierre de envíos: **4:30 p.m.** (hora del servidor; hay cuenta regresiva en la página).

### Permitido

- Cualquier lenguaje, librería o framework de código abierto (Ultralytics, MMDetection, Detectron2, torchvision, etc.).
- Pesos preentrenados públicos (ImageNet, COCO, DOTA, etc.).
- Cómputo propio: portátiles, Google Colab, Kaggle Notebooks u otros servicios gratuitos.
- Data augmentation, imágenes sintéticas generadas a partir de train, pseudo-etiquetado automático del conjunto oculto con su propio modelo.
- Usar asistentes de programación con IA.

### No permitido

- **Usar datos etiquetados externos**, en particular las colecciones originales (FAIR1M, ShipRSImageNet, VHRShips, xView): contienen las imágenes del conjunto oculto.
- Etiquetar a mano, o buscar las etiquetas de, las imágenes del conjunto oculto.
- Compartir código, predicciones o modelos entre equipos, o enviar con el token de otro equipo.
- Intentar vulnerar el servidor del leaderboard.

### Verificación

Los equipos deberán mostrar a los jueces su código o notebook y reproducir su envío final, o explicar cómo lo obtuvieron. Un equipo que no pueda demostrarlo, o que haya roto las reglas, queda descalificado y sube el siguiente. Las decisiones del comité organizador son definitivas.

## 7. Agenda del día (propuesta)

| Hora | Actividad |
|---|---|
| 7:30 – 8:00 | Registro, conformación de equipos y entrega de tokens |
| 8:00 – 8:15 | Inauguración (FAC + PUJ) |
| 8:15 – 9:15 | Webinars introductorios por parte del equipo de trabajo |
| 9:15 | Explicación del reto |
| 10:30 | Inicio del hackatón · se abre el leaderboard |
| 12:30 – 13:30 | Almuerzo (se puede seguir trabajando) |
| 16:30 | Cierre de envíos |
| 16:30 – 17:30 | Preparación de videos |

| Hora | Actividad |
|---|---|
| 18:00 | Cierre del hackathon |

*Tabla 4. Agenda tentativa; los horarios pueden ajustarse.*

# 8. Cómo empezar

1. **Forme su equipo** y reciba la tarjeta con su clave para enviar.
2. **Abra uno de los baselines** en Google Colab o Kaggle (active la GPU T4): `baseline_colab.ipynb` (YOLO26) o `baseline_frcnn_colab.ipynb` (Faster R-CNN).
3. **Cargue los datos:** suba `spacelab_dataset.zip` a su Drive, o agréguelo como Input en Kaggle.
4. **Ejecute el notebook.** Entrena el modelo, calcula el puntaje oficial en test, elige el umbral y genera el ZIP de envío.
5. **Suba el ZIP** en la sección Enviar del leaderboard y vea su posición.
6. **Mejore.** Ese es el reto.

**Los baselines son solo puntos de partida** para verificar que su flujo funciona de punta a punta. No son una recomendación: pueden usar cualquier modelo o framework (MMDetection, Detectron2, torchvision, RT-DETR, cualquier versión de YOLO, etc.). Si usa otro, formato_envio.py convierte sus cajas al formato oficial y arma el ZIP.

## Evaluar localmente

```
python evaluar.py --gt spacelab_dataset/labels/test --envio mis_preds.zip --barrer-umbral
```

Imprime el puntaje oficial, el desglose por clase y por fuente, y el umbral que maximiza el F1.

## Ideas para superar el baseline

- **Resolución y tiles:** los barcos son diminutos. Entrene a más resolución o recorte las imágenes grandes en mosaicos (SAHI).
- **Modelos más grandes** (YOLO26s/m, RT-DETR, etc.) si el tiempo de GPU lo permite.
- **Augmentations para vista cenital:** rotaciones de 90°, flips verticales, mosaic, cambios de color.
- **Desbalance:** work_vessel tiene pocos ejemplos; pruebe sobremuestreo o pérdidas ponderadas.
- **Memoria de GPU:** algunas imágenes tienen cientos de barcos y disparan el consumo de memoria al entrenar. Use lotes (batch) pequeños, sobre todo con modelos grandes o mayor resolución.
- **Inferencia:** TTA, ensambles con Weighted Boxes Fusion y ajuste del umbral. Nota: si van a usar TTA, tengan en cuenta que con YOLO26 la opción augment=True de Ultralytics no tiene efecto (la ignora con un aviso y predice normal).
- **Gestione su tiempo:** una sesión gratuita de Colab puede desconectarse. Guarde checkpoints en Drive y envíe temprano.

# Anexo · Glosario

| Término | Significado |
|---|---|
| IoU | Intersección sobre unión entre dos cajas: 1 si coinciden exactamente, 0 si no se tocan. |
| VP / FP / FN | Verdadero positivo (acierto), falso positivo (falsa alarma), falso negativo (barco no detectado). |
| Precisión | De lo que el modelo marcó, qué fracción era correcta. |
| Recall | De los barcos reales, qué fracción encontró el modelo. |
| F1 | Media armónica de precisión y recall. |
| AP / mAP | Área bajo la curva precisión–recall de una clase / promedio sobre clases (y aquí también sobre umbrales de IoU). |
| Confianza | Número entre 0 y 1 con el que el modelo indica qué tan seguro está de una detección. |
| Umbral | Confianza mínima para considerar una detección como "alarma" en el cálculo del F1. |